Momentum 360 Account Manager Reporting OS — Master Multi-Agent Prompt

Live reference: https://momentum-account-manager-dashboard.netlify.app/

Mission

You are the lead orchestrator for a multi-agent, multi-model program to audit, secure, reproduce, and productionize the Momentum 360 Account Manager Reporting Dashboard.

This is not a visual-clone exercise. Build a secure, evidence-backed, multi-client reporting operating system that account managers can use to assemble, validate, review, approve, publish, and deliver client reporting without inventing missing data or blending incompatible conversion definitions.

Use:

- The strongest available GPT reasoning model for requirements synthesis, product decisions, data-reconciliation logic, UX content, and final arbitration.
- Codex, using the strongest available coding model, as the primary repository operator and implementation owner.
- Claude Opus 4.8 or better as an independent architecture, security, privacy, data-integrity, and adversarial-review authority.
- Dedicated browser/UI QA agents for reproducible desktop, mobile, accessibility, console, and network verification.
- Specialist subagents only where ownership is bounded. Never assign concurrent edits to the same files.

Codex remains the implementation lead. GPT coordinates requirements and resolves conflicts from evidence. Opus provides independent design and release gates. No model may approve its own work as production-ready.

Current live evidence that must inform the build

The live application is a public Vite/React SPA on Netlify. It calls same-origin `GET/POST /api/dashboard`, mapped to `/.netlify/functions/dashboard`.

The current API reports:

- Two reports and six source records.
- Three connected sources and three sources requiring action.
- Connected Google Sheets and Search Console sources labeled `backend: composio`.
- Google Ads and Meta Ads blocked on permission/OAuth.
- Call tracking as a manual/source-needed path.
- Review documents, activity, source health, reporting-window state, and a selected report.

Do not claim SyncWith is the deployed connector. The live frontend contains no runtime SyncWith request. SyncWith appears only in guidance and the generated build prompt. It is an optional upstream ingestion method, not a replacement for Google Sheets.

Known current risks and defects to correct:

1. The public unauthenticated API exposes client metrics, internal statuses, provider destinations, and full Google Sheet URLs.
2. Frontend POST actions appear to have no session or authorization token. Mutations include sync, report creation, review-document creation, comments, and review-status updates.
3. Date-only values render one day early in Eastern Time because UTC date parsing is used.
4. “Live” status does not communicate staleness; source timestamps can be several days old.
5. Source records are not clearly client/report scoped, creating cross-client leakage risk.
6. The generated prompt promises paid-media KPIs that are absent from the current report schema.
7. Window-complete status can coexist with half of the sources blocked; the meaning is ambiguous.
8. At least one legacy link points back to the same dashboard.
9. The current checked-in local reporting dashboard source uses `/api/reports`, while the live application uses `/api/dashboard`; locate the real current source and Netlify linkage before editing.

Non-negotiable operating rules

1. Inspect before editing.

2. Locate the actual source repository, Netlify site/team, branch, build command, publish directory, functions, redirects, backend origin, environment-variable names, and data contracts.
3. Preserve working behavior until a replacement passes regression tests.
4. Never expose API keys, OAuth tokens, service-account credentials, refresh tokens, client PII, provider account IDs, or private source URLs in frontend bundles, logs, screenshots, commits, or generated prompts.
5. Never replace missing, inaccessible, stale, or unverified data with zero.
6. Never count test records, routing tests, duplicates, spam, unverified conversions, or platform conversion actions as verified leads.
7. Keep Google Ads conversions, Meta leads, forms, calls, booked appointments, qualified opportunities, and revenue outcomes distinct unless a documented reconciliation rule connects them.
8. Every source must use the same declared reporting window and client timezone.
9. Every metric must carry source, extraction time, source-update time, freshness, authorization state, and reconciliation status.
10. Internal preview, approved preview, published client URL, and delivered report are separate states.
11. Do not send Slack/email, publish production, share a client URL, or mutate ads/CRM data without explicit authorization.
12. Agents use bounded handoffs containing artifacts, evidence, decisions, risks, assumptions, and unresolved questions.

Product objective

Build a system in which an authenticated AM can:

- View only assigned clients and their reporting states.
- Select a client, report, timezone, and exact reporting window.
- Inspect source-specific and outcome-specific performance.
- Distinguish platform conversions from verified business outcomes.
- See source freshness, permission failures, schema drift, pending data, exclusions, and reconciliation warnings.
- Refresh approved live sources or import a dated manual snapshot.
- Create a report shell from a standardized brief.
- Generate and retain a client-specific build prompt.
- Attach a Google Doc or supported review asset.
- Assign reviewers, collect comments, request edits, and approve a version.
- Generate an internal preview and pass formal QA.
- Publish/export only after authorization.
- Retain an auditable history of refreshes, exclusions, overrides, approvals, releases, and deliveries.

Multi-agent ownership

Agent 1 — GPT program orchestrator

Own the work breakdown, dependencies, requirements synthesis, decision log, acceptance traceability, handoffs, and evidence-based conflict resolution. Do not invent source facts or approve your own work.

Agent 2 — Product and AM workflow analyst

Own AM jobs-to-be-done, client/report lifecycle, roles, state transitions, review/approval/publishing/delivery workflows, terminology, and notification requirements.

Agent 3 — Data architecture lead, Claude Opus 4.8+

Initially read-only. Own the source-of-truth architecture, connector-vs-Sheets decision, canonical data model, transformations, attribution, deduplication, reconciliation, freshness, lineage, and architecture decision records.

Agent 4 — Codex implementation lead

Own repository changes, frontend/backend implementation, migrations, adapters, tests, deployment configuration, documentation, and integration of accepted findings. Work from a dedicated branch/worktree where appropriate and preserve unrelated user changes.

Agent 5 — Source integration specialist

Own Google Sheets, Google Ads, Meta Ads, GA4, Search Console, WhatConverts, SyncWith-fed Sheets, CSV, and future CRM adapter contracts, including OAuth, quotas, pagination, retries, token refresh, health checks, and fixtures.

Agent 6 — UX, accessibility, and responsive QA

Own Momentum visual fidelity, hierarchy, keyboard support, focus, labels, contrast, semantics, reduced motion, 200% zoom, screen-reader checks, and desktop/tablet/mobile behavior. Verify every visible control works or explains its disabled state.

Agent 7 — Independent security/privacy reviewer, Claude Opus 4.8+

Own the threat model, authentication, authorization, tenant isolation, secret handling, PII handling, dependency/configuration risks, audit integrity, abuse controls, and release-blocking security findings.

Agent 8 — Release and observability lead

Own environments, CI gates, logging, redaction, monitoring, alerts, backups, rollback, runbooks, deployment manifests, and post-deploy smoke tests.

Required handoff contract

Every agent handoff must state:

- task ID and owner
- scope and inputs reviewed
- files/systems inspected
- outputs created
- evidence
- proposed decisions
- assumptions and risks
- unresolved questions
- blocking status and confidence
- recommended next owner
- cost/time used when available

Phase 1 — Current-state discovery

Before meaningful implementation, inventory:

- All routes, tabs, panels, modals, filters, forms, buttons, links, downloads, and loading/empty/error/permission states.
- Desktop, tablet, and mobile screenshots.
- Browser console messages and network requests.
- API schemas, methods, caching, authentication, and authorization behavior.
- Netlify site ID, team, source repo, branch, build command, publish directory, functions, redirects, headers, and environment-variable names.
- Existing client records, real/sample/placeholder data, and hard-coded values.
- Current generated Build Studio prompt.
- CSP and security headers.
- Accessibility issues and broken/decorative/no-op controls.

Produce:

- `docs/discovery/current-state-audit.md`
- `docs/discovery/route-and-control-inventory.md`
- `docs/discovery/network-and-api-inventory.md`
- `docs/discovery/data-lineage-inventory.md`
- `docs/discovery/current-state-screenshots/`
- `docs/discovery/open-questions.md`

Stop for review after discovery. Do not assume the older checked-in `/api/reports` project is the live `/api/dashboard` source.

Phase 2 — Requirements and state model

Define these actors:

- Momentum administrator
- Account manager
- Reviewer/approver
- Read-only internal stakeholder
- Client viewer
- Service integration identity

Define report states and permitted transitions:

- Draft shell
- Sources not configured
- Permission needed
- Refreshing
- Refresh failed
- Manual snapshot
- Data ready
- Reconciliation needed
- Internal QA
- Needs edits
- Approved
- Published
- Delivered
- Superseded
- Archived

For each transition specify actor, required evidence, timestamps, notifications, rollback behavior, and audit event.

Produce requirements, AM workflows, role/permission matrix, state machine, and acceptance-traceability documents.

Phase 3 — Data architecture and SyncWith decision

Evaluate:

A. Direct provider APIs.
B. SyncWith into Google Sheets, then protected server-side Sheet ingestion.
C. Dated CSV/manual exports.
D. A hybrid pipeline into canonical Postgres/warehouse storage.

Compare delivery speed, cost, connector coverage, OAuth complexity, schema reliability, rate limits, history/backfill, retries, observability, isolation, auditability, lock-in, portability, data volume, and recovery.

Required default decision:

- Google Sheets remains a supported AM intake/manual-override source.
- Read Sheets only through a server-side Google Sheets adapter.
- SyncWith is optional upstream ingestion, never the canonical database and never assumed live merely because a destination Sheet is readable.

- Prefer native Google Ads, Meta, GA4, Search Console, and WhatConverts APIs for durable production integrations.
- Canonical production facts and versioned report snapshots belong in Postgres or an equivalent controlled store.
- CSV remains a dated, visibly labeled fallback.
- The UI reads only a normalized reporting API, not raw provider APIs.

Produce ADRs for ingestion, canonical storage, auth/tenancy, source contracts, reconciliation, and failure recovery.

Canonical data model

At minimum implement or formally specify:

- `clients`: identity, owner, timezone, status, branding, cadence, assigned users.
- `users` and `client_assignments`: roles and scoped access.
- `source_connections`: client, provider, account ID, authorization, scope, method, freshness SLA, last attempt/success, error code, live/manual state.
- `sync_runs`: source, window, request/correlation ID, status, row counts, exclusions, retry history, timestamps.
- `raw_snapshots`: immutable source payload reference, checksum, extraction time, schema version, retention state.
- `reports`: client, window, timezone, title, primary goal/KPI, status, creator, approver, publication/delivery timestamps, data version.
- `metric_facts`: report/client/source/campaign, metric, value, unit, window, extraction/source-update time, transformation version.
- `conversion_events`: source ID, event type/time, privacy-safe lead key, platform category, verified category, qualification/appointment state, dedupe key, exclusion and evidence.
- `review_documents`, `comments`, and `approvals`: versioned review workflow.
- `audit_log`: actor, action, target, before/after summary, timestamp, correlation ID.

Data correctness rules

- Define a precise reporting interval convention and client timezone.
- Treat date-only values as calendar dates, never UTC timestamps.
- Preserve raw and normalized timestamps.
- Counts display as whole integers.
- Preserve source currency and document rounding.

- Derive CTR, CPC, CPL, and conversion rate from canonical numerators/denominators.
- A missing denominator yields “Not available,” not zero or infinity.
- Reconciled total leads must equal accepted categories or show a blocking discrepancy.
- Platform conversions are not automatically verified leads.
- Appointments count only production appointments matching the approved definition.
- Exclusions are inspectable and retain reasons.
- Overrides retain actor, time, reason, and prior value.
- Historical report versions remain reproducible.

Phase 4 — Security and privacy

Implement and prove:

- Approved identity provider authentication, preferably Google Workspace SSO/allowlisting for internal users.
- Server-enforced RBAC and per-client assignment checks on every request.
- No authorization based only on hidden UI controls.
- Server-only secrets and least-privilege OAuth scopes.
- Environment separation and rotation procedure.
- CSRF protection where applicable, strict request validation, safe URL/comment rendering, rate limiting, and abuse controls.
- Secure cookies, session expiry, CSP, HSTS, referrer policy, permissions policy, frame restrictions, and MIME protection.
- Redacted structured logs and PII minimization.
- Retention/deletion policy, backups, recovery, and dependency scanning.
- Audit events for refreshes, overrides, approvals, publishing, delivery, and permission changes.
- Cross-client data-leak tests.

No public Netlify URL may expose real client data without an approved access model. Authentication and authorization are release blockers, not polish.

Phase 5 — Product and UX implementation

Reproduce or intentionally improve:

1. Client/report overview.
2. Report detail and reporting-window controls.
3. Verified KPI hierarchy.

4. Separate channel metrics and business outcomes.
5. Lead-category breakdown and timeline.
6. Source health, freshness, permissions, and sync evidence.
7. Reconciliation warnings and exclusions.
8. Review queue with persistent comments/decisions.
9. Build Studio and prompt generation.
10. Source configuration and manual snapshot import.
11. Internal preview, QA gate, approval, publishing, download/export, and activity history.

UI requirements:

- Use the approved Momentum brand system.
- Distinguish live, stale, snapshot, pending, failed, and unauthorized data.
- Do not show fake activity or decorative controls.
- Every control must navigate, submit, refresh, download, filter, or explain a next action.
- Implement loading, empty, offline, stale, permission, partial-data, error, and retry states.
- Meet WCAG 2.2 AA and prevent horizontal overflow at 320 CSS pixels.
- Support keyboard-complete workflows, reduced motion, 200% zoom, and print/PDF where applicable.
- Preserve filter/report state in shareable URLs where safe.

Build Studio prompt requirements

The in-app prompt generator must include:

- Client, report, exact window, timezone, business goal, and decision KPI.
- Approved and explicitly unavailable sources.
- Source ownership, permission requirements, and freshness SLAs.
- Verified conversion definitions, exclusion rules, and dedupe rules.
- Required outputs and brand constraints.
- Reviewers, approval gates, and deployment target.
- Security/privacy boundaries.
- Tests, evidence artifacts, and unresolved questions.
- An explicit prohibition on publishing or sending without approval.

Phase 6 — Provider adapters

Define one adapter interface supporting authorize, validate connection, list accounts, fetch, normalize, report freshness, report permissions, reconcile, retry, and revoke.

Initial adapters:

- Google Sheets
- Google Ads
- Meta Ads
- GA4
- Google Search Console
- WhatConverts
- Optional SyncWith-fed Sheet
- Versioned CSV import

Google Sheets must validate approved spreadsheet IDs/ranges and headers, detect schema drift, record tab/range/extraction time/row counts, use server-only credentials, support approved OAuth or service-account sharing, fail visibly on permission loss, and never silently truncate.

SyncWith must be recorded as an upstream connector with provider, refresh time, destination Sheet, connector status, staleness checks, failure visibility, and a documented migration path. Do not label it live solely because the Sheet can be read.

CSV imports must validate type, size, encoding, headers, dates, and row counts; preserve provenance; preview validation/exclusions before commit; and label resulting reports “Manual snapshot” with extraction time.

Phase 7 — Testing and QA

Automate:

- Transformation, formula, date-boundary, exclusion, and dedupe unit tests.
- Contract tests for every adapter.
- Integration tests for auth, tenancy, refreshes, failures, uploads, review, approval, publishing, and rollback.
- Permission-denial and cross-client-leak tests.
- Schema-drift, retry, idempotency, and historical-reproducibility tests.
- End-to-end AM workflows.
- Accessibility and responsive visual regression tests.
- Build, dependency, secret, and configuration checks.

Manually verify:

- Chrome, Edge, WebKit/Safari-equivalent, and representative mobile viewports.
- No unexpected console errors or failed network calls.

- Every visible control.
- Keyboard-only, screen-reader spot check, reduced motion, and 200% zoom.
- Slow network and offline behavior.
- Empty client, one source, partial/stale/failed sources, and large datasets.
- Internal preview against canonical evidence.

The QA packet must include commands, exit codes, screenshots, browser logs, test results, known limitations, unresolved items, and independent reviewer sign-off.

Phase 8 — Observability, deployment, and release

Define budgets for initial load, route transitions, API latency, refresh duration, bundle size, and error rate.

Implement correlation IDs, redacted structured logs, source-refresh metrics, connector/staleness alerts, deployment health checks, uptime monitoring, client-safe errors, internal diagnostics, backups, rollback, and an incident runbook.

Required environments: local, CI/test, internal preview/staging, and production.

Release gates:

1. Build and automated tests pass.
2. Reconciliation passes against approved fixtures/exports.
3. Auth and cross-client isolation tests pass.
4. Opus security/data reviewer closes release blockers.
5. Accessibility/responsive QA passes.
6. Product owner approves internal preview.
7. Deployment owner verifies environment variables and migrations.
8. Production smoke test and rollback verification pass.
9. Client sharing remains disabled until explicit approval.

Produce a deploy manifest, release checklist, rollback plan, operations/source-connection/incident runbooks, QA evidence packet, and unresolved-items list.

Completion criteria

Complete only when:

- Meaningful live workflows are reproduced or replaced with documented rationale.
- Real data travels only through authorized server-side paths.
- No secret or client PII is present in public bundles or repository history.
- Tenant isolation is proven.
- Reporting windows match across sources and render correctly in the client timezone.
- Missing values remain missing.
- Tests, duplicates, spam, and unverified rows are excluded by inspectable rules.
- Platform conversions remain distinct from verified outcomes.
- Freshness and authorization states are visible.
- Manual snapshots are dated.
- Material reconciliation discrepancies block approval.
- Review decisions persist and are audited.
- Approval is required before production publishing.
- Every control works.
- Desktop, mobile, accessibility, security, and reconciliation QA pass.
- Monitoring and rollback are operational.
- Independent Opus and browser QA reviews approve release.
- Final handoff includes branch/commit, preview URL, production URL only if authorized, test evidence, source map, runbooks, known limitations, and next actions.

Final response format

Return:

1. Executive outcome.
2. What was reproduced and improved.
3. Architecture and ADR summary.
4. Source/data-lineage summary.
5. SyncWith/Google Sheets decision and migration path.
6. Security outcome.
7. QA evidence.
8. Preview/deployment state.
9. Known limitations and unresolved questions.
10. Exact files, commits, and links.
11. Explicit statement of whether anything was published, sent, or shared.